Lab 4

DC Motors: Interfacing and Control

Ronald Nguyen

Nguyen.rona@northeastern.edu

Ayaan Gupta

Gupta.ayaa@northeastern.edu

Professor Thomas R. Consi

EECE 2160, Section 2

Submit Date: October 24, 2026

Due Date: October 24, 2026

Abstract

The purpose of this lab was to design a digital circuit on the DE1-SoC board that generates an RC PWM signal to control the angle of an RC servo. The desired angle was entered on the slide switches, limited to 180°, and shown on the 7-segment displays using the display_8bits circuit from Lab 2. LPM multiplier and adder blocks converted the angle to a pulse width in clock periods, and an LPM counter and comparator generated the PWM signal. Pulse widths measured on an ADALM 2000 oscilloscope at 0°, 90°, and 180° were compared with the expected values, and the circuit was demonstrated to the TA.

Introduction

An RC servo is not a basic motor, but a brushed DC motor inside a feedback position control system that sets the angle of its output shaft. The desired angle is sent as an RC PWM signal, where the width of each pulse sets the position instead of delivering power to the motor. A 0.6 ms pulse moves the servo to 0°, 1.5 ms moves it to 90°, and 2.4 ms moves it to 180°. The DE1-SoC outputs 3.3 V logic, so the signal must be shifted to 5 V for the servo to read it reliably.

The FPGA measures time in cycles of the 50 MHz clock, which are 0.02 µs each. For the prelab, the number of clock periods for the counter to roll over every 20 ms is:

$$20\text{ ms} / 0.02\ \mu\text{s} = 1{,}000{,}000 \text{ clock periods}$$

The count values at the minimum, middle, and maximum pulse widths are:

$$0.6\text{ ms} / 0.02\ \mu\text{s} = 30{,}000 \text{ clock periods } (0°)$$

$$1.5\text{ ms} / 0.02\ \mu\text{s} = 75{,}000 \text{ clock periods } (90°)$$

$$2.4\text{ ms} / 0.02\ \mu\text{s} = 120{,}000 \text{ clock periods } (180°)$$

Since the 180° range spans 120,000 − 30,000 = 90,000 clock periods, each degree adds 90,000 / 180 = 500 clock periods, so an angle θ in 1° increments maps to a pulse width N of:

$$N = 500\theta + 30{,}000, \quad 0° \leq \theta \leq 180°$$

Section 3 of the lab specified a 50 ms (20 Hz) period, which only changes the rollover count to 50 ms / 0.02 µs = 2,500,000 clock periods. The pulse widths depend on time, not the period, so the equation above still applies.

Setup

This lab was completed in Quartus Prime Lite with an Intel DE1-SoC board. The PWM signal was output on the GPIO 1 header (JP2) and sent through a 3.3 V to 5 V logic level shifter on a solderless breadboard, using input LV4 and output HV4, to the servo's signal wire. The level shifter's low side was referenced to the board's 3.3 V and ground pins, and its high side to the

breadboard's 5 V rail. The servo was powered from an external 5 V supply on the breadboard rails instead of the board. The PWM signal was also sent to a second pin so it could be viewed on channel 1 of an ADALM 2000 oscilloscope using Scopy. The connections are listed in Table 1. The display_8bits files from Lab 2 were reused, and the LPM_COUNTER, LPM_COMPARE, LPM_MUX, LPM_CONSTANT, LPM_MULT, and LPM_ADD_SUB components were added from the IP Catalog.

Table 1: Connections between the DE1-SoC, the level shifter, the RC servo, and the ADALM 2000

Signal	FPGA Pin	JP2 Pin	Connected To
GPIO1_35 (PWM)	PIN_AC22	40	Level shifter LV4
GPIO1_34 (Scope)	PIN_AA20	39	ADALM 2000 CH1
+3.3 V	—	29	Level shifter LV
Ground	—	30	Level shifter GND (LV side)
Ground	—	12	ADALM 2000 GND
Servo signal	—	—	Level shifter HV4
+5 V (external)	—	—	Servo +V, level shifter HV
Ground (external)	—	—	Servo GND, level shifter GND (HV side)

Design

The circuit, shown in Figure 1, has three parts: an input limiting section, an angle to pulse width conversion, and a PWM generator.

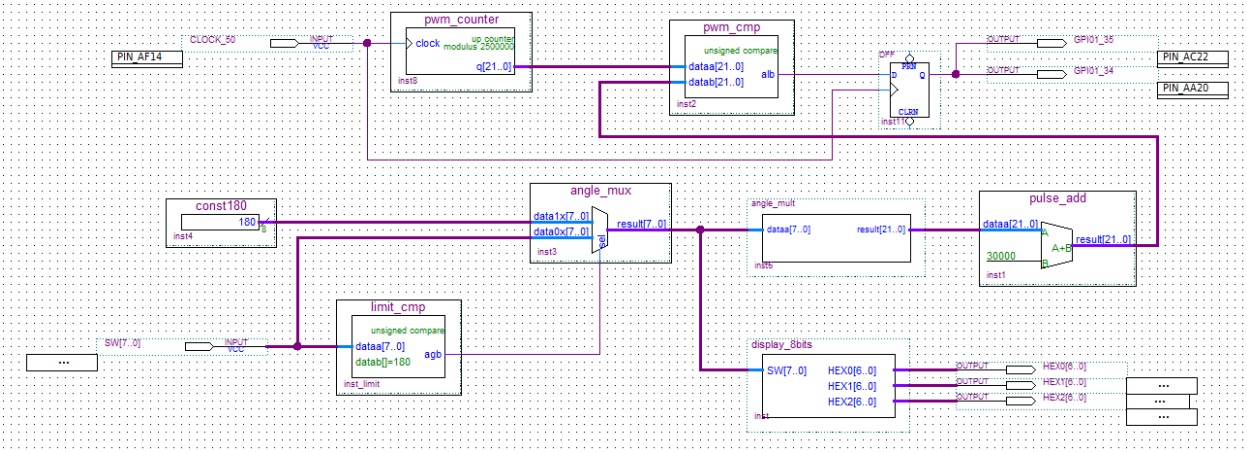


Figure 1: Schematic of the RC servo circuit

Limiting the Input Angle

The 8 slide switches, SW[7..0], allow angles up to 255, so the input had to be limited to 180. Limit_cmp is an 8-bit unsigned LPM_COMPARE whose agb output is high when SW is greater than a constant 180. This drives the select of angle_mux, a 2:1 LPM_MUX that passes SW on data0x when agb is low and the output of const180, an 8-bit LPM_CONSTANT, on data1x when agb is high. Its output, angle[7..0], stays at 180 for any input above 180. The angle is also sent to the display_8bits block from Lab 2 and shown on HEX0 through HEX2. As in Lab 3, display7 outputs segment a on O[6], so the HEX pins were assigned in reverse order from the DE1-SoC pin table.

Converting the Angle to a Pulse Width

Following the prelab equation, angle_mult, an unsigned LPM_MULT, multiplies the angle by a constant 500. Its constant input was made 14 bits wide so the product is 22 bits and matches the rest of the circuit. Pulse_add, an LPM_ADD_SUB set to add only, adds a constant 30,000 to give

pulse[21..0] = 500θ + 30,000. This ranges from 30,000 clock periods (0.6 ms) at 0° to 120,000 clock periods (2.4 ms) at 180°.

Generating the PWM Signal

Pwm_counter is a 22-bit LPM_COUNTER clocked by CLOCK_50 with a modulus of 2,500,000, so it rolls over every 50 ms and sets the PWM frequency at 20 Hz. 22 bits were needed since 2,500,000 is larger than the 2,097,152 that 21 bits can hold. Its output feeds pwm_cmp, an LPM_COMPARE whose alb output is high while the count is below pulse. This creates a pulse of 500θ + 30,000 clock periods at the start of each period. The alb output is registered by a D flip-flop clocked by CLOCK_50 to remove any glitches from the comparator, and its Q output drives both GPIO1_35 and GPIO1_34.

Results & Analysis

Debugging the Circuit

When first compiled, Quartus gave syntax errors and errors saying modules were declared more than once. The _inst.v and _bb.v files created by the IP Catalog wizard had been added to the project along with the main files, and removing them from the project fixed the errors. After the counter was widened to 22 bits, the angle_mult symbol kept its old 20-bit output, so its symbol file had to be regenerated before it matched pulse_add. When the board was first programmed, the display showed an upside-down A instead of 0, the same reversed segment order seen in Lab 3. The HEX pins had been assigned from the DE1-SoC pin table, and flipping the assignments fixed the display. When the servo was first powered, it spun continuously instead of moving to a set angle, which matched the behavior of a continuous rotation servo, where pulse width sets speed and direction instead of position. The servo was replaced, but breadboard wiring issues

unrelated to the schematic logic continued to affect it during testing. The TA reviewed the circuit and approved the demonstration.

Pulse Width Measurements

Scopy was set to 1 V/div, triggering on the rising edge of channel 1, with the measure tool showing the period and positive pulse width. Before the servo was powered, the switches were set to 0°, 90°, and 180°, and the pulse width at each angle was measured as shown in Figures 2 through 4. The 0° capture was taken at 1 ms/div and the 90° and 180° captures at 10 ms/div. The measured widths were compared with the prelab values and the displayed angle in Table 2.

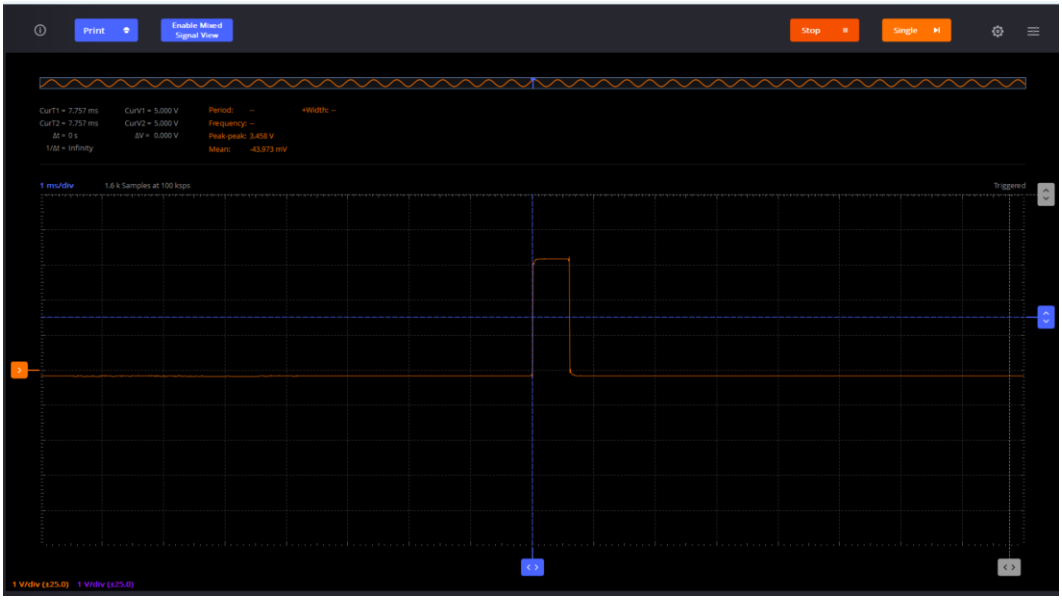


Figure 2: PWM signal at 0°, 1 ms/div, pulse width ≈ 0.6 ms

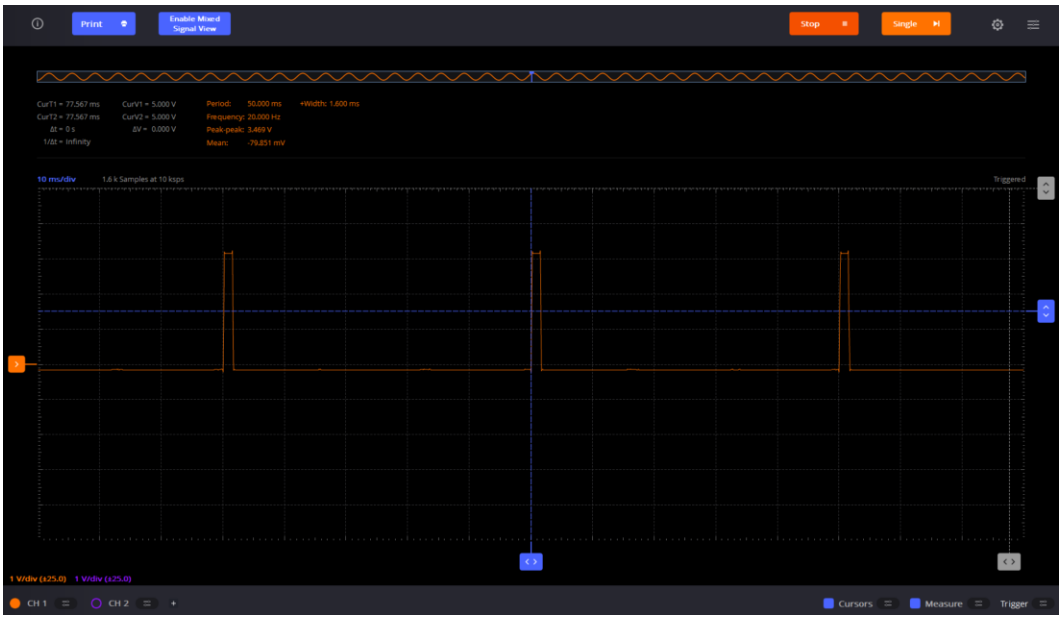


Figure 3: PWM signal at 90°, period = 50.000 ms, +Width = 1.600 ms

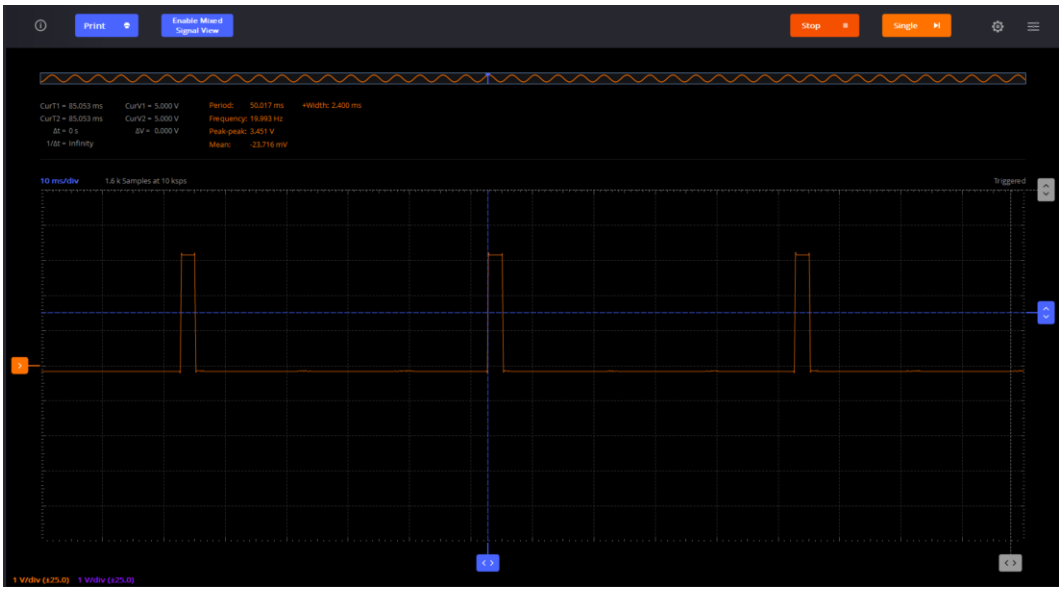


Figure 4: PWM signal at 180°, period = 50.017 ms, +Width = 2.400 ms

Table 2: Comparison of expected, measured, and displayed values

Angle (°)	Expected Width	Measured Width	Error	Displayed (°)
0	0.6 ms	≈ 0.6 ms	≈ 0%	0
90	1.5 ms	1.600 ms	6.7%	90
180	2.4 ms	2.400 ms	0.0%	180

The measured period was 50.000 ms at 90° and 50.017 ms at 180°, within 0.04% of the 50 ms design. The 0° and 180° widths matched the expected values, and the 90° width was 0.1 ms high. At 10 ms/div, Scopy samples at 10 ksps, so each sample is 0.1 ms apart and the measured width can only be accurate to about one sample. This means the 90° error came from the scope's resolution, not the circuit. The 0° capture was taken at 1 ms/div, where the samples are 0.01 ms apart, but only one pulse was on screen, so the measure tool could not calculate the period or width, and the width was read from the grid instead. The signal measured about 3.45 V peak-to-peak since the scope pin is taken before the level shifter, matching the board's 3.3 V logic. Photos of the servo at set angles were not captured due to time, so Figures 2 through 4 serve as the record of the output at each angle.

Conclusion

An RC PWM signal for controlling an RC servo was successfully generated with the DE1-SoC using a counter and comparator, with the pulse width set by the angle on the slide switches through the prelab equation $N = 500\theta + 30{,}000$. Inputs above 180 were limited to 180 on both the display and the PWM output. The measured pulse widths matched the expected 0.6 to 2.4 ms range within the oscilloscope's resolution, and the period matched the 50 ms design. Viewing the signal on the oscilloscope before connecting the servo confirmed the circuit was correct, which separated circuit problems from servo problems.

References

[1] "Lab 4, DC Motors: Interfacing and Control," Northeastern University, EECE 2160, Fall 2026.